AI 388 / CS 388 Natural Language Processing

Module 5: Transformers and Decoding

Topics 14, 15, 18

Fall 2026 midterm study guide

Built from the lecture videos/transcripts, slides, Canvas exercises, Assignments 1–3 and the Fall 2022/2023 midterms.

Prepared October 2026 · lecture notation throughout

How to use this guide

Each **Topic** number is the item number on the official Fall 2026 midterm topic list. Section numbers are *topic.section*, so "2.4" is Topic 2, Section 4. Every topic has the same layout:

- **In plain words**: the idea in one paragraph.
- **Key boxes**: definitions and formulas in **lecture notation**.
- **Steps**: the algorithm or exam procedure, step by step.
- **Worked examples**: full calculations (checked numerically with Python).
- **Past exam**: every relevant Fall 2022/2023 midterm question and Canvas exercise, with the official answer and an explanation. Official errata are noted.
- **Full derivations** (added Oct 20): every formula derived step by step with nothing assumed, plus low-dimensional by-hand examples of the algorithms worked in lecture and "conceptual checks" for Viterbi/CKY.
- **Traps**: places where convention differences or common slips cost points.
- **Practice**: original questions, with answers right after.

Notation (lecture): input $\mathbf{x}$; feature vector $f(\mathbf{x}) \in \mathbb{R}^d$; weights $\mathbf{w}$; label $y \in \{-1, +1\}$ (binary) or $y \in \mathcal{Y}$ (multiclass); step size α; D training examples $(\mathbf{x}^{(i)}, y^{(i)})$; log is the natural log unless a question says otherwise. Perceptron decision rule: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (a score of 0 is negative). Where a past exam used a different convention, the box says so in red.

Exam reminders. Auto-graded short answers must match the requested format exactly (rounding, fractions vs. decimals, list syntax like `[w1, w2]`). Adding explanation text makes the answer wrong. Long answers: show work concisely for partial credit and box final answers.

Contents

How to use this guide **1**

14 Transformers: Architecture Details **3**

14.1 One Transformer layer (encoder block) . . . 3
14.2 One full layer by hand (low dimension, verified numerically) . . . 4
14.3 Parameter counting . . . 5
14.4 Using Transformers (lecture 5.0.1) . . . 5
14.5 A3 Part 1: letter counting . . . 6
14.6 Efficiency extensions (lecture 5.0.3, MC fodder) . . . 6
14.7 Practice . . . 6

15 Transformer Language Modeling (Assignment 3) **8**

15.1 Model . . . 8
15.2 Training . . . 8
15.3 The causal mask . . . 9
15.4 Inference and scoring (A3 `NeuralLanguageModel`) . . . 9
15.5 Evaluation: perplexity on A3 . . . 9
15.6 Your A3 model, piece by piece . . . 9
15.7 Full derivations . . . 10
15.8 Past exam problems . . . 11
15.9 Practice . . . 12

18 Decoding: Greedy, Beam Search, Nucleus Sampling **13**

18.1 Exhaustive vs. greedy vs. beam . . . 13
18.2 Sampling and nucleus sampling . . . 14
18.3 Derivations . . . 14
18.4 Past exam: Fall 2023 Q1(16–18) . . . 15
18.5 Practice . . . 15

TOPIC 14

Transformers: Architecture Details

Sources: Lectures 5.0.0 Transformer Architecture, 5.0.1 Using Transformers, 5.0.3 Transformer Extensions; slides seg-trans1, seg-trans2, seg-trans4; A3 Part 1; Fall 2023 Q1(9–11).

In plain words

A Transformer layer = **multi-head self-attention** (tokens exchange information) + a **position-wise feed-forward network** (each token is processed on its own), each wrapped with a **residual connection** (add the input back) and **layer normalization** (rescale). Every layer maps n vectors of size d_{model} to n vectors of size d_{model}, so layers stack. Attention gets the headlines, but most parameters and compute live in the feed-forward networks.

14.1 One Transformer layer (encoder block)

Block structure (Vaswani et al. 2017, "post-norm" as drawn in lecture)

$$\mathbf{H} = \text{LayerNorm}\big(X + \text{MHSA}(X)\big)$$
$$\text{Out} = \text{LayerNorm}\big(\mathbf{H} + \text{FFN}(\mathbf{H})\big), \qquad \text{FFN}(\mathbf{h}) = W_2\,\text{ReLU}(W_1\mathbf{h} + \mathbf{b}_1) + \mathbf{b}_2$$

- **FFN is applied to each position independently** (same weights for every token). It's a plain one-hidden-layer network $d_{model} \to d_{ff} \to d_{model}$ (lecture calls it $d_{internal}$).
- **Residual connections**: add the sublayer's input to its output. Gradients flow easily and the model can effectively "skip" layers it doesn't need. This forces sublayer outputs to have dimension d_{model}.
- **Layer normalization**: rescale each token's vector (mean 0, variance 1, then learned scale/shift). It keeps layers on the same scale so one learning rate works for deep stacks.
- **Input:** token embeddings + positional encodings (Topic 13). **Output:** contextualized vectors, one per token.

Dimensions

quantity	symbol	Vaswani base / notes
model (residual stream) width	d_{model}	512 (input = output of each block)
query/key size per head	d_k	64 (must match between Q and K)
value size per head	d_v	64 (concat hd_v, then W^O back to d_{model})
heads	h	8 ($hd_k = d_{model}$)
FFN hidden	d_{ff}	2048 ($4 \times d_{model}$)
layers	N	6

GPT-3 scaling: d_{model} grows to about 12,288, but d_{head} stays at 128. In OPT-175B, MHSA is about 17% of the FLOPs and the **FFN about 80%**. The huge FFNs are thought to store much of the model's factual knowledge.

14.2 One full layer by hand (low dimension, verified numerically)

Worked example: A3 Part 1-style layer: attention → residual → Linear–ReLU–Linear → residual

Two tokens, $d_{model} = d_k = 2$, one head, no LayerNorm (A3 Part 1 says you likely don't need it). $X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (token 1 is "a", token 2 is "b"), $W^Q = W^K = W^V = I$, bidirectional attention.

1. $Q = K = V = X$. Scores $QK^\top/\sqrt{2} = \begin{bmatrix} 0.7071 & 0 \\ 0 & 0.7071 \end{bmatrix}$.
2. Row softmax: $e^{0.7071} = 2.0281$, so row $1 = (2.0281, 1)/3.0281 = (0.6698, 0.3302)$. $A = \begin{bmatrix} 0.6698 & 0.3302 \\ 0.3302 & 0.6698 \end{bmatrix}$.
3. Attention output $AV = A$ (because $V = I$). Each token keeps 67% of itself and mixes in 33% of the other.
4. **Residual 1:** $H = X + AV = \begin{bmatrix} 1.6698 & 0.3302 \\ 0.3302 & 1.6698 \end{bmatrix}$.
5. **FFN, row by row** (same weights for both tokens): $W_1 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$ ($d_{ff} = 2$), $W_2 = 0.5I$, no biases. Token 1: $W_1\mathbf{h}_1 = (1.3395, -1.3395)$; ReLU $\to (1.3395, 0)$; $W_2 \cdot \to (0.6698, 0)$. Token 2: $(-1.3395, 1.3395) \to (0, 1.3395) \to (0, 0.6698)$.
6. **Residual 2:** Out $= H + \mathrm{FFN}(H) = \begin{bmatrix} 2.3395 & 0.3302 \\ 0.3302 & 2.3395 \end{bmatrix}$.

Shapes stayed 2×2 throughout ($n \times d_{model}$), which is what lets layers stack. Only step 2 mixed information across tokens; step 5 treated each row separately.

Layer normalization, the formula

For one token's vector $\mathbf{h} \in \mathbb{R}^d$: $\mu = \frac{1}{d}\sum_j h_j$, $\sigma^2 = \frac{1}{d}\sum_j (h_j - \mu)^2$,

$$\mathrm{LN}(\mathbf{h})_j = \gamma_j \frac{h_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_j \qquad (\gamma, \beta \text{ learned, length } d;\ \epsilon \text{ tiny, e.g. } 10^{-5}).$$

Example ($\gamma = 1, \beta = 0, \epsilon \approx 0$): $\mathbf{h} = (2, 4, 6)$: $\mu = 4$, $\sigma^2 = \frac{4+0+4}{3} = 2.667$, $\sigma = 1.633$, so LN $= (-1.2247, 0, 1.2247)$. It normalizes *each token separately, across its features* (batch norm normalizes each feature across the batch). Curiosity: with $d = 2$ any (a, b) with $a \neq b$ becomes $(\pm 1, \mp 1)$, e.g. $(1.6698, 0.3302) \to (1, -1)$. That's why hand examples usually skip LN.

Derivations

1. **Why residuals help gradients.** $\mathbf{y} = \mathbf{x} + F(\mathbf{x})$ gives $\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = I + \frac{\partial F}{\partial \mathbf{x}}$. Even if F's Jacobian is tiny, the identity term passes the gradient straight through. Through L stacked blocks the product of Jacobians always contains the identity path, so it doesn't vanish the way the RNN product does (Topic 12). If F learns to output 0, the block is skipped.
2. **Why dimensions must match.** $\mathbf{x} + F(\mathbf{x})$ is only defined if $F(\mathbf{x}) \in \mathbb{R}^{d_{model}}$. Hence W^O maps $hd_v \to d_{model}$ and the FFN ends with $d_{ff} \to d_{model}$.

3. **FLOPs per token** (one multiply-add per weight used): Q, K, V, O projections $4d_{model}^2$; FFN $2d_{model}d_{ff} = 8d_{model}^2$ when $d_{ff} = 4d_{model}$; attention scores and weighted sum $2n\,d_{model}$ (with $hd_k = d_{model}$). FFN share for short sequences: $8/(4+8) = 2/3$; it grows as d_{model} grows relative to n, consistent with OPT's 80%.
4. **[CLS] vs. averaging.** Averaging gives every token weight $1/n$, fixed. The [CLS] output is $\sum_j A_{\text{CLS},j}\mathbf{v}_j$ with *learned, input-dependent* weights, so the model can focus on the tokens that matter for the task.

14.3 Parameter counting

Per layer (biases and LayerNorm in parentheses)

- MHSA: W^Q, W^K, W^V: $3 \cdot d_{model} \cdot hd_k$; W^O: $hd_v \cdot d_{model}$. With $hd_k = hd_v = d_{model}$: $\mathbf{4d_{model}^2}$ $(+4d_{model})$.
- FFN: $\mathbf{2\,d_{model}\,d_{ff}}$ $(+d_{ff} + d_{model})$.
- 2 LayerNorms: $(4d_{model})$.
- Plus embeddings $|V|d_{model}$, learned positions $n_{max}d_{model}$, output layer $d_{model}|V|$ (often tied to the embeddings).

Vaswani base: $4(512^2) = 1{,}048{,}576$ for attention, $2(512)(2048) = 2{,}097{,}152$ for the FFN. The FFN is twice the attention, and that's 3,145,728 per layer (3,152,384 with biases + LN).

Midterm Fall 2023 Q1(9)–(11)

(9) A "3-layer Transformer" whose middle layer has only Multi-Head Attention + Add&Norm (its FFN removed). Parameters vs. a standard 3-layer?

Official answer: C. Fewer (one FFN missing). (10) Performance across many tasks?

Official answer: C. Worse.
(11) A 2-layer Transformer where each layer is MHA + FFN + FFN (two FFNs) vs. a standard **4-layer**:

Official answer: C. Fewer. 2 MHA + 4 FFN vs. 4 MHA + 4 FFN: same number of FFNs, but two fewer attention blocks.

14.4 Using Transformers (lecture 5.0.1)

- **API:** a sequence of vectors in, a same-length sequence of contextualized vectors out (like RNN states).
- **Per-token classification** (POS tagging, A3 Part 1 letter counting): linear + softmax on each output vector.
- **Sequence classification** (sentiment): either average the outputs then linear + softmax (DAN-style), or (standard) prepend a **[CLS]** token and classify from its output vector. [CLS] attends to every token, so it learns a task-specific weighting. That's smarter than average pooling.
- **Sentence-pair classification** (entailment): feed both sentences in one sequence. Self-attention aligns words across the two (*driving* vs. *walking*). This was a headline BERT result (Topic 16).
- The original Transformer is **encoder–decoder** (for translation). Language models use a decoder-only stack with a causal mask (Topic 15).

14.5 A3 Part 1: letter counting

Task: for each of 20 characters, predict how many times that character occurred **before** (0, 1, or $\geq$2): 3 classes at every position. Example: "i like movies a lot" $\rightarrow$ `00010010002102021102`. The BEFOREAFTER variant counts both directions.

- Implement from scratch (only Linear, softmax, nonlinearities): **single-head attention** $\rightarrow$ **residual** $\rightarrow$ **Linear–ReLU–Linear** $\rightarrow$ **residual**. LayerNorm is optional for a shallow net. Queries/keys/values come from Linear layers; use matmul + transpose; divide by $\sqrt{d_k}$.
- Output: log-probabilities $[20 \times 3]$ + the attention maps; NLLLoss over all positions at once.
- **Positional encodings needed** for BEFORE (without them the model sees a bag of characters and can't tell before from after). They're learned `nn.Embedding` positions added to character embeddings: $\text{emb}_{char}(t) + \text{emb}_{pos}(0)$.
- Targets: $>$ 95% (reference $>$ 98% with 1–2 layers). If stuck at about 70%, layers stored in a plain Python list aren't registered (use `nn.ModuleList`).

14.6 Efficiency extensions (lecture 5.0.3, MC fodder)

- **Scaling laws** (Kaplan et al. 2020): test loss falls as a smooth power law (log-linear) in compute, data and parameters. Transformers keep improving with scale, unlike RNNs.
- **Quadratic attention** is the bottleneck. **Performer** multiplies $K^\top V$ first (a linear-time approximation, not the same computation). **Longformer** uses sparse attention patterns (sliding window, dilated, global). These mostly save *memory*, not time, and approximations usually underperform full attention.
- Much recent progress is systems work (GPU memory handling) for longer contexts.

Traps

- "Transformer" $\neq$ "self-attention": a block also has the FFN, residuals and layer norm.
- The FFN is position-wise: it does *not* mix information across tokens. Only attention does.
- Residuals require matching dimensions, so the MHSA output (after W^O) and the FFN output must be d_{model}.
- d_k can be smaller than d_{model}; W^Q/W^K needn't be square.
- Removing a sublayer removes parameters. Count MHSA blocks and FFN blocks separately (Fall 2023 Q9–11).

14.7 Practice

Practice

1. $d_{model} = 256$, $h = 4$, $d_k = d_v = 64$, $d_{ff} = 1024$, 6 layers, vocabulary 10,000, untied output, no biases/LN/positions. Total parameters?
2. Which costs more FLOPs per token in a large Transformer: attention projections or the FFN? Why?
3. A model replaces each FFN with a second MHSA block. More or fewer parameters with the standard $d_{ff} = 4d_{model}$?

Answers

1. Per layer $4(256^2) + 2(256)(1024) = 262{,}144 + 524{,}288 = 786{,}432$; $\times 6 = 4{,}718{,}592$; embeddings + output $2 \times 10{,}000 \times 256 = 5{,}120{,}000$; total 9,838,592.
2. The FFN ($2d\,d_{ff} = 8d^2$ per token vs. $4d^2$ for the projections), plus attention's n^2 term for long sequences.
3. Fewer: an MHSA block is $4d^2$ vs. $8d^2$ for the FFN.

TOPIC 15

Transformer Language Modeling (Assignment 3)

Sources: Lectures 5.0.2 Transformer Language Modeling, 4.0.2 LM Evaluation; slides seg-trans3, seg-lm3; Assignment 3 Part 2; Fall 2023 Q1(13–14).

In plain words

Feed the text into a Transformer and, at **every** position, predict the *next* token from that position's output vector (linear layer + softmax over the vocabulary). Training uses all positions at once: inputs are the sequence, targets are the same sequence shifted left by one, and the loss is the summed (or averaged) negative log-probability of the true next tokens. A **causal mask** stops a position from attending to future tokens. Otherwise the model just copies the answer. Evaluate with perplexity on held-out text.

15.1 Model

Next-token distribution

With $\mathbf{h}_i$ the final-layer vector at position i:

$$P(w_{i+1} = w \mid w_{1:i}) = \frac{\exp(\mathbf{w}_w \cdot \mathbf{h}_i)}{\sum_{w' \in V} \exp(\mathbf{w}_{w'} \cdot \mathbf{h}_i)}$$

equivalently $P(\cdot \mid w_{1:i}) = \text{softmax}(W\mathbf{h}_i)$ with $W \in \mathbb{R}^{|V| \times d_{model}}$. Rows of W act like (output) word embeddings: the model puts mass on words compatible with $\mathbf{h}_i$.

15.2 Training

Shifted targets, simultaneous prediction

1. Sequence "I saw the dog". Input = $\langle s \rangle$ I saw the; targets = I saw the dog. (A3: **space is the start symbol**. Feed space + first 19 chars and predict all 20.)
2. Run the Transformer **with a causal mask**. The output is $[n, |V|]$ log-probabilities (batched: $[B, n, |V|]$).
3. Loss = $\sum_i -\log P(\text{target}_i \mid \text{prefix})$ = NLL, i.e. the log-likelihood of the whole chunk (log of a product = sum of logs). `nn.NLLLoss(log_probs [n,|V|], targets [n])`. NLLLoss averages by default.
4. Batched: reshape $[B, n, |V|] \to [Bn, |V|]$ and targets $[B, n] \to [Bn]$ before NLLLoss.
5. zero_grad → forward → loss → backward → step (Adam).

Batching (lecture): split the stream into chunks (e.g. length 5 or 20) and stack independent chunks along a batch dimension. That is *not* the same as training on one long sequence: each chunk sees only its own tokens, so positions and attention never span chunks.

15.3 The causal mask

Why and how

Why: without it, position i attends to $i+1$, copies the next token into its representation, and predicts it perfectly. Perplexity drops near 1 and the A3 sanity check fails. That isn't language modeling: the model must represent $P(w_i \mid w_1, \ldots, w_{i-1})$.
How: before the softmax, add M with $M_{ij} = -\infty$ for $j > i$ (future) and 0 otherwise. Then $e^{-\infty} = 0$, so future tokens get zero attention.

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix} \qquad A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)$$

PyTorch: `torch.triu(torch.ones(n,n), diagonal=1)` marks the future (True ⇒ `masked_fill(-inf)`), or pass a triangular $0/-\infty$ `mask` to `nn.TransformerEncoder`. Row 1 of A is always $[1, 0, 0, \ldots]$: the first token attends only to itself.

15.4 Inference and scoring (A3 `NeuralLanguageModel`)

- `get_next_char_log_probs(context)`: input = space + context (truncate to the last n_{max} positions). Return the **last row** of log-probabilities, a full length-27 vector of natural logs. **Not** an argmax: evaluation needs the probabilities.
- `get_log_prob_sequence(next_chars, context)` = $\sum_k \log P(c_k \mid \text{context}, c_{1:k-1})$: score each character *before* appending it to the context.
- Call `model.eval()` (turns off dropout) and use `torch.no_grad()`.
- **Normalization check:** each returned distribution must sum to 1. Range check: probabilities in $[0, 1]$.

15.5 Evaluation: perplexity on A3

A3 numbers

Dev set 500 characters, 27 types. **Uniform model:** total log-prob $500 \ln(1/27) = -1644$, average -3.296, perplexity $= e^{3.296} = 27$. Exponentiating the average log-probability gives $1/27$, the *inverse* of perplexity. Requirement: PPL ≤ 7 in under 10 minutes; reference 6.3. **Your run** (4 layers, $d_{model} = 32$, $d_{internal} = 32$, chunks of 20, Adam 10^{-3}, 10 epochs): log-prob -864.02, average -1.728, **PPL** $= e^{1.728} = 5.63$.

$$\text{PPL} = \exp\left(-\tfrac{1}{N}\textstyle\sum_i \ln P(c_i \mid c_{<i})\right).$$

15.6 Your A3 model, piece by piece

```
x = embedding(indices)              # [T] -> [T, d_model]   (27 x 32 table)
x = positional_embedding(x)         # + learned position vectors (20 x 32)
for block in blocks:                # 4 x TransformerBlock
```

```
    a = causal_attention(x)        # Q,K: Linear(32,32); V: Linear(32,32); /sqrt(d_k); triu mask
    x = LayerNorm(x + a)           # residual + norm
    x = LayerNorm(x + FF2(ReLU(FF1(x))))
logits = output_layer(x)           # [T, 27]
log_probs = log_softmax(logits, dim=1)    # NLLLoss with targets [T]
```

Single-head attention without W^O (since V already maps to d_{model}). Total parameters with biases: 24,027 (5,408 per block).

15.7 Full derivations

A. The training loss is exactly the log-likelihood of the chunk

1. Chain rule: $P(w_1..w_n) = \prod_{i=1}^{n} P(w_i \mid w_{<i})$ (exact, no Markov assumption).
2. $-\log P(w_1..w_n) = \sum_i -\log P(w_i|w_{<i})$. Each term is one position's NLL.
3. With a causal mask, the output at position i only depends on inputs $\leq i$, so the row used for target w_{i+1} really is a function of $w_{\leq i}$ only. All n terms come from one forward pass.
4. NLLLoss's default `mean` divides by n: same gradient direction, step scaled by $1/n$.

B. Why a missing mask gives perplexity $\to 1$

1. Without the mask, row i's attention can put weight on position $i+1$, whose input embedding *is* the target.
2. With positional encodings the model can learn "attend to the next position" and copy that token's identity through V to the output layer.
3. Then $P(\text{target}_i) \to 1$ at every position, so the average NLL $\to 0$ and PPL $= e^0 = 1$. The A3 handout: "perplexity will get very close to 1 very quickly".

C. Scoring a continuation, step by step

1. $\log P(c_1..c_m \mid \text{ctx}) = \sum_{k=1}^{m} \log P(c_k \mid \text{ctx}, c_1..c_{k-1})$ (chain rule).
2. So: for $k = 1..m$, call `get_next_char_log_probs(ctx + `$c_{1..k-1}$`)`, read entry c_k, add it, then append c_k.
3. Equivalent and faster: one forward pass over space + ctx + $c_{1..m-1}$ and read the last m rows (each at its target index). Same numbers, because of the causal mask.

Worked example: the output layer, by hand

Take the two final vectors from Topic 14's worked layer, $\mathbf{h}_1 = (2.3395, 0.3302)$, $\mathbf{h}_2 = (0.3302, 2.3395)$, and a 3-word vocabulary with output rows $\mathbf{u}_x = (1,0)$, $\mathbf{u}_y = (0,1)$, $\mathbf{u}_z = (1,1)$.

1. Logits for position 1: (2.3395, 0.3302, 2.6698).
2. log-softmax: $\log \sum e^{\ell} = \ln(10.376 + 1.391 + 14.437) = \ln 26.204 = 3.2659$, so log-probs $= (-0.9264, -2.9357, -0.5962)$; probs $(0.396, 0.053, 0.551)$.
3. If the true next token at position 1 is x: NLL $= 0.9264$. If position 2's target is y: also 0.9264 (symmetric). Average NLL 0.9264, PPL $= e^{0.9264} = 2.525$.

D. A3 Part 1 labels, derived for the handout example

Label at position $i = \min\big(2,\ \#\{j < i : c_j = c_i\}\big)$. String "i like movies a lot␣" (20 characters, trailing space).

1. i(0) ␣(0) l(0) i(1) k(0) e(0) ␣(1) m(0) o(0) v(0) i(2) e(1) s(0) ␣(2) a(0) ␣(3→2) l(1) o(1) t(0) ␣(4→2).
2. Concatenate: `00010010002102021102`, matching the handout. BEFOREAFTER counts all *other* occurrences: $\min(2, \text{count}(c) - 1)$, giving `22120120102102021102`.
3. How attention can do it: let token i's query match keys of the same character (e.g. $W^Q = W^K$ on one-hot characters, scaled up). With a causal-style restriction to $j < i$ plus position information, the attention mass on matching earlier positions encodes the count, which the FFN maps to $0/1/2$. Without positional encodings BEFORE is impossible (self-attention has no notion of order, Topic 13), but BEFOREAFTER is fine, because it doesn't depend on order.

A3 chunking option (handout): you may prepend a few extra context tokens to each chunk and *not* compute loss on them, so every scored prediction has real context. Minor gain.

15.8 Past exam problems

Midterm Fall 2023 Q1(13)

Possible attention-map shapes when training an A3 LM:

Official answer: II. [seq len, seq len]; III. [batch size, seq len, seq len].

Midterm Fall 2023 Q1(14)

Possible shape for the model outputs fed to the loss:

Official answer: V. [batch size, seq len, vocab size]. Not [batch, vocab] (the model predicts at *every* position), not [batch, seq, d_{model}] (hidden states before the output layer). Strictly, NLLLoss wants it flattened to $[B \cdot n, |V|]$, but V is the model's output shape.

Worked example: causal attention on 3 tokens

Unscaled scores $S = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 3 \end{bmatrix}$ (from Topic 13). Masked: $\begin{bmatrix} 1 & -\infty & -\infty \\ 1 & 1 & -\infty \\ 2 & 1 & 3 \end{bmatrix}$. Row-softmax: $A = \begin{bmatrix} 1 & 0 & 0 \\ 0.5 & 0.5 & 0 \\ 0.245 & 0.090 & 0.665 \end{bmatrix}$. Token 3 (the last) is unaffected, since it can already see everything.

Traps

- **Off-by-one:** the output at position i predicts token $i + 1$. Inputs start with the start symbol, and targets are the inputs shifted by one.
- The causal mask uses $-\infty$ *before* softmax (not 0 after). Masking with 0 in score space still gives weight $e^0 = 1$.
- Perplexity uses natural log consistently (A3 returns base-e logs). Average over the number of *predicted* characters.

- Return log-probabilities, not probabilities, when the API says log.
- The LM output is $[\ldots, |V|]$, not $[\ldots, d_{model}]$.

15.9 Practice

Practice

1. A chunk "hello" in A3's setup: write the input and target strings.
2. Your LM assigns the 4 true next characters probabilities 0.5, 0.2, 0.1, 0.4. Total log-probability, average NLL, perplexity?
3. A classmate's model reaches dev perplexity 1.02 after one epoch. Most likely bug?
4. Batch of 16 chunks of length 20, vocabulary 27. Give the shapes of the input indices, the output log-probs, the targets, and what you pass to NLLLoss.

Answers

1. Input "␣hell", target "hello".
2. $\ln 0.5 + \ln 0.2 + \ln 0.1 + \ln 0.4 = -0.6931 - 1.6094 - 2.3026 - 0.9163 = -5.5215$; average NLL 1.3804; PPL $= e^{1.3804} = 3.976$.
3. A missing or incorrect causal mask: the model sees the future.
4. $[16, 20]$; $[16, 20, 27]$; $[16, 20]$; flatten to $[320, 27]$ and $[320]$.

TOPIC 18

Decoding: Greedy, Beam Search, Nucleus Sampling

Sources: Lectures 5.1.0 Beam Search, 5.1.1 Nucleus Sampling; slides seg-infer1, seg-infer2; Exercise 5.0; Fall 2023 Q1(16–18).

In plain words

A trained LM gives a next-token distribution. Turning that into text needs a *decoding strategy*. **Greedy**: always take the top token. **Beam search**: keep the k best partial sequences at each step, which finds higher-probability outputs; good for translation. **Sampling**: draw at random, for variety, but the long tail of junk tokens eventually gets sampled. **Nucleus (top-p) sampling**: sample only from the smallest set of top tokens covering probability p, renormalized. Varied but sane; it's what chat models typically use.

18.1 Exhaustive vs. greedy vs. beam

- Exhaustive search over length-n outputs: $|V|^n$ sequences, exponential.
- **Greedy:** $\hat{w}_t = \arg\max_w P(w \mid \hat{w}_{<t})$. Cheap and works OK, but can get derailed by an early locally-good choice.

Beam search with beam size k

1. Step 1: score all $|V|$ first tokens; keep the top k.
2. Each step: extend every beam hypothesis by every token, giving $k|V|$ candidates scored by **total sequence probability** (product, or sum of logs).
3. Keep the top k candidates overall (not top k per parent). Repeat until length n / end-of-sequence.
4. Return the best (or all k, if asked).

Cost: about kn Transformer calls (one per beam item per step) and $k|V|n$ hypotheses considered, vs. $|V|^n$ for exhaustive. $k = 1$ is greedy.

When to use: conditional generation with one right answer (machine translation). Less important for very large LMs, and expensive. **Degeneration:** beam search on open-ended text falls into repetition loops ("Universidad Nacional Autónoma de México/…" repeated). Each step is locally likely, and the loop has high overall probability.

Worked example: beam beats greedy

$P(w_1)$: A 0.5, B 0.4, C 0.1. $P(\cdot|A)$: x 0.4, y 0.3, z 0.3. $P(\cdot|B)$: x 0.9, y 0.05, z 0.05. Greedy: A then x, giving $0.5 \times 0.4 = 0.20$. Beam $k = 2$: keep {A, B}. Candidates: Ax 0.20, Ay 0.15, Az 0.15, Bx 0.36, By 0.02, Bz 0.02. Keep {Bx 0.36, Ax 0.20}, so the best is **B x** (0.36), which beats greedy.

18.2 Sampling and nucleus sampling

Nucleus (top-p) sampling (Holtzman et al. 2020)

1. Sort tokens by probability, high to low.
2. Keep the smallest prefix whose cumulative probability is $\geq p$ (the "nucleus").
3. **Renormalize** within the nucleus (divide by the kept mass) and sample.

Why: if the tail holds 10% of the mass, pure sampling picks a tail word about every 10 tokens and goes off the rails. Results: nucleus sampling has moderate perplexity (not as low as greedy/beam) but far less repetition (low self-BLEU), and good human-judged quality.

Canvas Exercise 5.0

Q1 true of nucleus sampling:

Answer: the most likely word *may* be chosen. Not "any word may be chosen" (the tail is cut), not "the most likely word *must* be chosen". Not "always introduces randomness at every step": if the top token alone has probability $\geq p$, it is chosen deterministically.
Q2: the: 0.7, a: 0.1, computer: 0.1, tablet: 0.08, phone: 0.01, server: 0.01, with $p = 0.95$. Cumulative: 0.7, 0.8, 0.9, **0.98** ≥ 0.95, so keep the first 4. $P(\text{tablet}) =$

Answer: $0.08/0.98 \approx 0.0816$.

18.3 Derivations

Numbers behind the lecture's claims

1. **Exhaustive count.** $|V|$ choices at each of n positions: $|V|^n$ sequences. $|V| = 50{,}000$, $n = 10$: $\approx 10^{47}$.
2. **Beam cost.** Step 1: one call, $|V|$ candidates, keep k. Each later step: k calls (one per kept prefix), $k|V|$ candidates, keep k. Total $\approx kn$ calls and $k|V|n$ candidates. Linear in n, not exponential.
3. **Greedy isn't optimal.** Greedy maximizes each factor separately. The product can be larger for a sequence whose *first* factor is smaller (worked example: $0.4 \times 0.9 > 0.5 \times 0.4$).
4. **The long tail, in numbers.** If the tail has mass 0.1 at every step, the expected number of tail tokens in m samples is $0.1m$, and $P(\text{no tail token in 10 steps}) = 0.9^{10} = 0.349$. So 65% of 10-token stretches contain at least one tail word.
5. **Renormalization is valid.** Kept set S with mass $Z = \sum_{w \in S} P(w) \geq p$. New $P'(w) = P(w)/Z$ for $w \in S$, 0 otherwise. Then $\sum_w P'(w) = Z/Z = 1$, and ratios between kept tokens are unchanged ($P'(a)/P'(b) = P(a)/P(b)$).
6. **Degeneration as arithmetic.** If a loop of L tokens has per-token probability q close to 1, repeating it r times costs only q^{Lr}. That can beat a fresh, varied continuation whose tokens each have probability around 0.1–0.3, so a probability-maximizing search prefers the loop.

18.4 Past exam: Fall 2023 Q1(16–18)

Midterm Fall 2023 Q1(16)–(18)

$P(a) = 0.8$, $P(the) = 0.15$, $P(dog) = 0.05$. $P(\cdot|a)$: a .01, the .01, dog .98. $P(\cdot|the)$: a .15, the .01, dog .84. $P(\cdot|dog)$: a .3, the .5, dog .2. Exactly two tokens are generated.
(16) Beam size 2. Step 1 keeps {a .8, the .15}. Step 2 candidates: a a .008, a the .008, **a dog .784**, the a .0225, the the .0015, **the dog .126**.

Official answer: a dog, the dog.
(17) How many sequences could pure sampling return? All 9 have nonzero probability:

Official answer: 9.
(18) Nucleus $p = 0.9$: step 1: $0.8 < 0.9$, add "the" to reach 0.95, so {a, the}. After a: dog $0.98 \geq 0.9$, so {dog}. After the: dog 0.84, add a to reach 0.99, so {dog, a}. Sequences: a dog; the dog; the a, so

Official answer: 3.

Traps

- Beam keeps the top k *overall* among all extensions, ranked by the product of probabilities so far, not just the last token's probability.
- Nucleus: include the token that *crosses* p ($\geq p$), then renormalize by the kept mass (0.98, not p=0.95).
- Nucleus is decided *per step*; the nucleus set changes with context.
- Count possible sequences path by path: each first token contributes the size of its own next-step nucleus (Q18: $1 + 2 = 3$), not 2×2.
- Greedy = beam with $k = 1$; beam with $k = |V|^n$ (unbounded) = exhaustive.

18.5 Practice

Practice

1. In the Fall 2023 setup, what does greedy decoding return, and with what probability?
2. Same setup, nucleus $p = 0.8$. How many sequences are possible?
3. Distribution: x .5, y .3, z .15, w .05, $p = 0.75$. What is the nucleus and the renormalized probability of y?
4. Beam size 3, $|V| = 50{,}000$, length 20. How many Transformer calls and hypotheses are scored (approximately)?

Answers

1. a, then dog: 0.784. 2. Step 1: a ($0.8 \geq 0.8$), so {a}. After a: {dog}. So 1 sequence. 3. Cumulative 0.5, $0.8 \geq 0.75$, so {x, y}; $P(y) = 0.3/0.8 = 0.375$. 4. About $kn = 60$ calls; $k|V|n = 3 \times 10^6$ hypotheses.